

```
marp: true
theme: default
paginate: true
size: 16:9
style: |
section { font-size: 28px; }
h1 { color: #1f4e79; }
h2 { color: #1f4e79; }
table { font-size: 22px; }
code { font-size: 20px; }
```

Certify AB

Verifierbara certifikat som tjänst i Azure

Problemet

- HR och utbildningsbolag delar ut kursintyg som Word-dokument
- Vem som helst kan ändra namnet och skriva ut
- **Ingen kan kontrollera om ett intyg är äkta**

Vår lösning

- Ett API som skapar certifikat med en **unik verifieringslänk**
- En arbetsgivare öppnar länken och ser direkt om intyget är äkta
- Ingen inloggning krävs för att verifiera

API:t

Endpoint	Vad den gör	Åtkomst
POST /certificates	Skapar certifikat, returnerar id och länk	Öppen
GET /certificates/{id}	Hämtar ett certifikat	Öppen
GET /verify/{uuid}	Publik verifiering	Öppen
GET /certificates	Listar alla	Kräver API-nyckel
GET /health	Visar att appen lever	Öppen

Dokumenterat i Swagger på /swagger

Arkitektur: så kommer ny kod ut

```
git push till GitHub
  ↓
Azure DevOps pipeline startar
  ↓
1. Bygg          kompilerar koden
  ↓
2. Test          kör testerna
  ↓
3. az acr build  bygger en image, sparas i ACR som certify-api:46
  ↓
4. Deploy        säger åt Container Appen: kör certify-api:46
  ↓
Container Appen hämtar imagen från ACR och startar 2 kopior
```

Misslyckas ett steg stannar pipelinen, och den gamla versionen fortsätter köra.

Arkitektur: när appen kör



- Containrar minns ingenting, så certifikaten sparas i **Blob Storage** som alla kopior delar
- Skapa certifikat: appen skriver en fil. Verifiera: appen letar efter filen
- All infrastruktur är skriven i **Bicep**, `infra/main.bicep`

Säkerhet: inga lösenord någonstans

Vem	Pratar med	Hur
Container App	ACR och Blob Storage	Managed Identity + RBAC-roller
Pipeline	Azure	Service connection med workload identity federation
API-nyckel	<code>GET /certificates</code>	<code>@secure()</code> Bicep-parameter, aldrig i git

- Blobbarna är **privata**, verifiering går alltid via API:t
- TLS 1.2 minimum, endast HTTPS

Live-demo

push → pipeline → live API-anrop

Demo: steg för steg

1. Liten ändring i koden, `git push` till main
2. Pipelinen startar av sig själv i Azure DevOps
3. Bygg, test, `az acr build`, deploy till Container Apps
4. Ny revision blir aktiv
5. `POST /certificates` i Swagger skapar ett certifikat
6. Öppna verifieringslänken, `isValid: true`

`https://certify-api.thankfulgrass-b2132c06.swedencentral.azurecontainerapps.io/swagger`

Ekonomi: månadskostnad vid lansering

40 kunder, 8 000 certifikat per månad, Sweden Central

Resurs	Kr/mån
Container Apps, 2 repliker dygnet runt	≈ 319
Container Registry, Basic	47,58
Blob Storage	≈ 1
Log Analytics	0
Totalt	≈ 367

Intäkt $40 \times 99 \text{ kr} = 3\,960 \text{ kr}$. Infrastrukturen är runt 9 procent.

Skalning: vad händer om länken går viralt?

- 10 000 delningar på en dag: $\approx$ **0,04 kr** i blob-läsningar
- Under 2 miljoner anrop per månad är gratis
- Autoskalning: **2 till 5 repliker** vid fler än 10 simultana anrop
- 3 gånger fler kunder: $\approx$ **370 kr**, nästan samma

Flaskhalsar: taket på 5 repliker, och `GET /certificates` som hämtar varje blob en och en

Åtgärd: cacha verifieringssvaret, ett certifikat ändras aldrig

Drift och designval

- **Autoskalning:** HTTP-regel, `concurrentRequests: 10` , max 5 repliker
- **Parametriserad Bicep:** `main.dev.bicepparam` 1 till 2 repliker,
`main.prod.bicepparam` 2 till 5
- **Rollback:** varje deploy blir en revision, tillbaka med `az containerapp update --image ...:45`

Vi valde	Framför	Varför
Container Apps	AKS	Ett API, inget driftteam
Privata blobbar	Publika länkar	Kan logga och spärra certifikat
<code>az acr build</code>	Docker-task	Samma connection, ingen Docker på agenten

Vad gick fel och hur vi löste det

1. Deploy fick **UNAUTHORIZED** mot ACR

Appen hade behörigheten, men visste inte att den skulle använda den.

Lösning: `registries` -block i Bicep med `identity: 'system'`, så appen visar sin Managed Identity mot ACR

2. Verifieringslänken fick **http://**

Azure avkrypterar trafiken innan den når appen, så appen visste inte att den var krypterad.

Lösning: vi tvingar `https://` när länken byggs i koden

Nästa steg

- Fler tester, bland annat mot lagringen med Azurite
- Cache framför `/verify/{uuid}`
- Application Insights med larm vid fel
- Autentisering per kund i stället för en gemensam API-nyckel

Frågor?

Repo: `github.com/josefalusseini/teamlab-molnlosningar`